```
BufferedReader br = new BufferedReader(
    new FileReader("notas.txt"));
while ((linea = br.readLine()) != null) {
    System.out.println(linea);
}
```



INVESTIGACIÓN · ALGORITMOS Y LÓGICA DE PROGRAMACIÓN

# Lectura de archivos y procesamiento de información en Java

Paquete java.io · FileReader, BufferedReader, FileWriter y BufferedWriter

**Grupo 5**

Sandra Alulema · Anthony Armas · Joselyn Guaman · Valentina Ortiz · Andrés Vayas

Universidad Técnica de Ambato · Ambato, Ecuador · Noviembre de 2026

# 1 ¿Por qué los programas necesitan archivos?

Cuando un programa en Java termina, todas sus variables y objetos desaparecen. Para que la información sobreviva (notas de estudiantes, registros de ventas, configuraciones) hay que guardarla en un **archivo**. Esta investigación explica qué es un archivo, cómo Java lo lee y lo escribe con las clases del paquete `java.io` y cómo se procesa la información que contiene.

> **Idea central**
> Leer es traer datos del archivo al programa, escribir es enviarlos del programa al archivo y almacenar es dejarlos guardados de forma permanente.

# 2 Archivos y sus tipos

Un archivo es un conjunto de datos relacionados que se guarda en un dispositivo con un nombre y una ubicación determinados. Según su contenido se agrupan en cuatro tipos:

| Tipo | Extensiones | Características |
|---|---|---|
| **Texto** | `.txt` `.csv` `.java` | Caracteres legibles; se abren con cualquier editor. |
| **Imágenes** | `.png` `.jpg` `.gif` | Almacenan datos gráficos. |
| **Documentos** | `.pdf` `.docx` `.xlsx` | Combinan texto, formato y otros elementos. |
| **Binarios** | `.exe` `.class` `.jar` | Guardan los datos como los entiende la máquina; no son legibles directamente. |

# 3 RAM y almacenamiento permanente

La memoria RAM guarda las variables y los objetos mientras el programa se ejecuta; el almacenamiento permanente (disco duro, SSD, memoria USB) conserva los datos aunque no haya energía.

| Característica | RAM | Almacenamiento permanente |
|---|---|---|
| **Persistencia** | Volátil: se pierde al cerrar el programa o apagar el equipo | Se conserva sin energía |
| **Velocidad** | Muy alta | Menor |
| **Uso en Java** | Variables y objetos | Archivos (.txt, .csv, etc.) |

Leer un archivo es copiar su contenido desde el almacenamiento permanente hacia la RAM. El proceso tiene tres pasos:

| 1. Abrir | → | 2. Recorrer | → | 3. Cerrar |
|---|---|---|---|---|
| Se abre el archivo | | Se lee su contenido | | Se libera el recurso |

# 4 El paquete java.io: rutas y clases

El paquete `java.io` reúne las clases de entrada y salida de Java; se importa con `import java.io.*;`. Para ubicar un archivo se usan rutas:

- **Ruta absoluta:** ubicación completa desde la raíz del sistema, por ejemplo `C:/Users/estudiante/datos/notas.txt`.
- **Ruta relativa:** parte de la carpeta del proyecto, por ejemplo `datos/notas.txt`. Es más portable porque funciona aunque el proyecto cambie de equipo.

| Clase | Para qué sirve |
|---|---|
| `File` | Representa una ruta de archivo o carpeta. Crear el objeto no crea el archivo en el disco, solo lo describe. |
| `FileReader` | Abre un archivo de texto y lo lee carácter por carácter. |
| `BufferedReader` | Se coloca sobre un FileReader, añade un búfer en memoria (lectura más eficiente) y habilita `readLine()`. |
| `FileWriter` | Escribe texto directamente en un archivo. |
| `BufferedWriter` | Envuelve un FileWriter, reduce los accesos al disco y ofrece `newLine()`. |

### Métodos principales de la clase `File`:

| Método | Qué hace |
|---|---|
| `exists()` | Indica si el archivo o la carpeta existe. |
| `createNewFile()` | Crea el archivo vacío; devuelve true si lo creó y puede lanzar IOException. |
| `delete()` | Elimina el archivo; devuelve true si lo logró. |
| `getPath()` | Devuelve la ruta con la que se creó el objeto. |

# 5 Leer archivos: read() y readLine()

| Método | Lee | Al llegar al final devuelve |
|---|---|---|
| `read()` | Un carácter a la vez (como entero) | `-1` |
| `readLine()` | Una línea completa como String | `null` |

Lectura de un archivo línea por línea con `try-with-resources`, que cierra el archivo automáticamente:

```
try (BufferedReader br = new BufferedReader(new FileReader("datos/notas.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    System.out.println("Error de lectura: " + e.getMessage());
}
```

## 6 Escribir archivos: sobrescribir o agregar

El método `write()` envía texto al archivo y `close()` cierra el flujo, lo que garantiza que los datos se guarden en el disco. El constructor decide qué pasa con el contenido previo:

| Modo | Código | Resultado |
|---|---|---|
| **Sobrescribir (por defecto)** | `new FileWriter("datos.txt")` | Borra lo que había y empieza de cero. |
| **Agregar (append)** | `new FileWriter("datos.txt", true)` | Conserva el contenido y añade al final. |

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter("materias.txt"))) {
    bw.write("Programación");
    bw.newLine();
    bw.write("Cálculo");
    bw.newLine();
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

**Cuidado**
Un error en el segundo parámetro de FileWriter puede borrar información valiosa. Revisa si lleva o no `true` antes de ejecutar.

Como las operaciones con archivos pueden fallar (ruta inexistente, falta de permisos, disco lleno), Java obliga a tratar la `IOException` con `try-catch` o con `throws`.

## 7 Entrada, proceso y salida

Todo programa que trabaja con archivos sigue el esquema Entrada, Proceso, Salida. Separar las tres etapas facilita el diseño del algoritmo, la prueba de escritorio y la depuración.

| Entrada | → | Proceso | → | Salida |
|---|---|---|---|---|
| Líneas leídas de notas.txt | | Convertir, sumar y calcular el promedio | | Promedio en pantalla y en resultados.txt |

# 8 Validación y conversión de datos

Los archivos pueden tener datos incompletos o incorrectos, por eso se validan antes de usarlos:

- Comprobar que el archivo exista antes de leerlo con `exists()`.
- Verificar que las líneas no estén vacías y que los valores estén en rango (por ejemplo, notas entre 0 y 10).
- Capturar excepciones con `try-catch` y mostrar mensajes claros en lugar de que el programa se detenga de golpe.

Todo lo que se lee de un archivo de texto llega como `String`. Para operar con números se convierte con `Integer.parseInt()` o `Double.parseDouble()`; si el texto no es numérico se lanza una `NumberFormatException` que debe capturarse. Ya convertidos, los datos se pueden buscar con `equals()` o `contains()` y filtrar, por ejemplo, las notas aprobadas.

```
String linea = "8.5";
try {
    double nota = Double.parseDouble(linea.trim());
    if (nota >= 7.0) {
        System.out.println("Aprobado");
    }
} catch (NumberFormatException e) {
    System.out.println("Dato no numérico: " + linea);
}
```

# 9 Conclusiones

- Los archivos hacen que la información persista más allá de la ejecución del programa, porque la RAM es volátil.
- Java organiza la lectura y la escritura en `java.io`: File describe el archivo, los Reader leen y los Writer escriben; las versiones con búfer son más eficientes.
- Elegir entre sobrescribir y agregar es una decisión de diseño importante.
- Excepciones, validación y conversión controlada hacen los programas robustos frente a archivos inexistentes, vacíos o con datos incorrectos.
- El esquema entrada, proceso y salida integra los archivos con el resto de la lógica de programación.

*Documento elaborado a partir del informe de exposición del Grupo 5.*